

PHYS7101: NUMERICAL METHODS AND DATA ANALYSIS · LECTURE 3

Solving equations and fitting models

Mathew Smith & Aneta Stefanovska

From numerical solutions to data

Where we left off

From Lecture 2

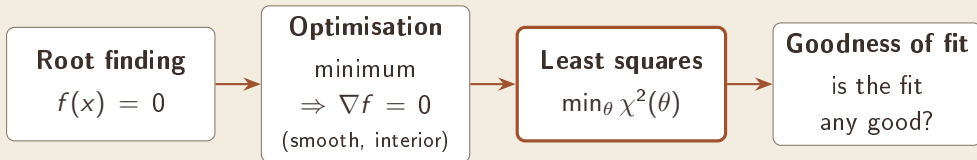
- Static problems need root finding
- Residuals $e_i = Y_i - \hat{X}(t_i)$
separate dynamics from noise
- **Verification:** did we solve the model correctly?
- **Validation:** does the model represent the system?

Today

- Solving $f(x) = 0$ robustly, in one and many dimensions
- Finding minima: optimisation
- Fitting models to data: least squares
- Judging a fit: residuals, χ^2 , uncertainties

Validation needs data. To compare a model with data, we must first **estimate its parameters** from the data.

One chain of ideas



The idea of today

Fitting a model is minimising a misfit. At a smooth interior minimum the gradient is zero, so the minimum is a root of the gradient — and root finding is where we start. Goodness of fit is the data-analysis check: after fitting the parameters, we still have to ask whether the model describes the data.

PART A

Solving equations

Root finding and optimisation: the engine inside every fit.

Bisection: slow but certain

If f is continuous and $f(a)f(b) < 0$, a root lies in $[a, b]$.

1. Take the midpoint $m = (a + b)/2$
2. Keep the half where the sign changes
3. Repeat

The bracket halves every step. To reach a tolerance ε :

$$n \approx \log_2 \frac{b - a}{\varepsilon}$$

Trade-off

Guaranteed to converge once you have a bracket.

Slow: one binary digit per step, about 33 steps for 10^{-10} .

Needs only the **sign** of f , not f' .

Newton–Raphson: solving $f(x) = 0$

Start from a guess x_n . Replace f by its tangent line at x_n and find where the tangent crosses zero:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Repeat until

$$|x_{n+1} - x_n| < \varepsilon.$$

Example: Kepler's equation

Where is a planet on its elliptical orbit at time t ?

- $M = 2\pi t/T$: mean anomaly, known from the clock
- e : eccentricity of the orbit
- E : eccentric anomaly, giving the position
 $x = a(\cos E - e)$, $y = b \sin E$

Given M and e , solve for E the equation

$$f(E) = E - e \sin E - M = 0,$$

with $f'(E) = 1 - e \cos E$. No closed form exists, so we iterate.

Newton–Raphson in action

Kepler's equation with $M = 1$, starting from $E_0 = M$. Error $|E_n - E_{\text{exact}}|$:

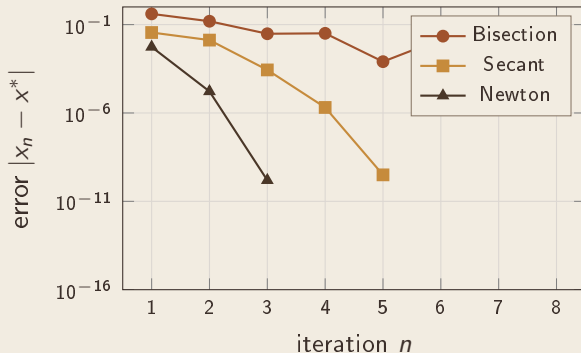
Iteration	$e = 0.5$	$e = 0.9$
1	8×10^{-2}	6×10^{-1}
2	2×10^{-3}	7×10^{-2}
3	6×10^{-7}	2×10^{-3}
4	9×10^{-14}	1×10^{-6}
5	—	4×10^{-13}

Near the root the number of correct digits roughly **doubles** each iteration (quadratic convergence).

It can fail

A poor guess, $f'(x) \approx 0$ or several roots can send it to the wrong root or to infinity. Check that $|f(x)|$ is small at the answer.

Bisection, secant and Newton compared



$f(x) = x^3 - 2x - 5$, root
 $x^* = 2.0945515 \dots$

Bisection	1 digit per step (binary)
Secant	order ≈ 1.6 , no f'
Newton	order 2, needs f'

Speed versus safety

Newton reaches machine precision in 4 steps; bisection is still at 10^{-3} after 8. But only bisection is guaranteed.

Robust root finding in practice

How Newton fails

- $f'(x_n) \approx 0$: a huge jump
- Starting far away: wrong root or divergence
- Cycling between two points
- No bracket: no guarantee a root exists

Brent's method

A hybrid: fast secant / interpolation steps, with a **bisection fallback** whenever a step leaves the bracket.

Guaranteed like bisection, usually as fast as secant.

`scipy.optimize.brentq(f, a, b)`

Bracket first (plot $f!$), then let a fast method finish.

Newton in several dimensions

Solve $F(x) = 0$ for $x \in \mathbb{R}^n$. Replace the derivative by the **Jacobian** $J_{ij} = \partial F_i / \partial x_j$:

$$J(x_n) \Delta x = -F(x_n), \quad x_{n+1} = x_n + \Delta x$$

Example

Intersection of $x^2 + y^2 = 4$ and $y = e^x$:

$$F = \begin{pmatrix} x^2 + y^2 - 4 \\ e^x - y \end{pmatrix}, \quad J = \begin{pmatrix} 2x & 2y \\ e^x & -1 \end{pmatrix}$$

What changes in n dimensions

Each step solves a **linear system** (never invert J). No bracketing exists, so the starting guess matters even more.
`scipy.optimize.root`

Optimisation: minima are roots of the gradient

At a smooth minimum $f'(x) = 0$ and $f''(x) > 0$, so Newton on f' gives

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

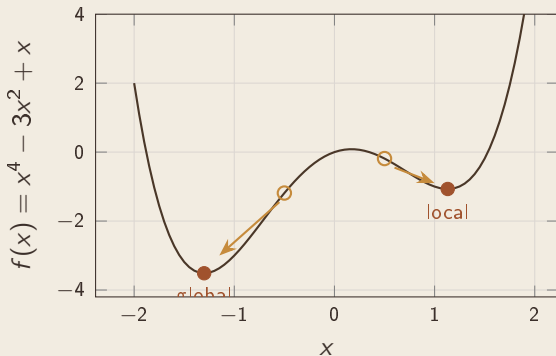
Golden-section search

The bracketing method for minima: keep three points with the lowest in the middle, shrink the interval by 0.618 each step. No derivatives; guaranteed for one minimum in the bracket.

Precision limit

Near a minimum $f(x) \approx f(x^*) + \frac{1}{2}f''(x^*)(x - x^*)^2$. Round-off in f of 10^{-16} limits x to about $\sqrt{10^{-16}} = 10^{-8}$.

Local and global minima



Gradient descent, $x_{n+1} = x_n - \eta f'(x_n)$:

- from $x_0 = -0.5$: $x \rightarrow -1.30$,
 $f = -3.51$
- from $x_0 = +0.5$: $x \rightarrow +1.13$,
 $f = -1.07$

Lesson

Every local method finds **a** minimum, not **the** minimum. Nonlinear problems can have several (Lecture 2). Try several starting points.

Minimising in many dimensions

Method	Needs	Character
Nelder–Mead (simplex)	f only	Robust, slow; noisy or non-smooth f
Gradient descent	∇f	Simple; slow in narrow valleys
Quasi-Newton (BFGS)	∇f (builds up f'')	The general-purpose default
Newton	∇f , Hessian	Fastest near the minimum; costly

In practice

`scipy.optimize.minimize(f, x0, method=...)`. The choice of x_0 is **yours**, and often matters more than the choice of method.

PART B

Fitting models

Choose the parameters that make the model closest to the data.

“Closest” must be defined — and it is a choice.

What is a fit?

Data $(x_i, y_i \pm \sigma_i)$, $i = 1, \dots, N$. Model $f(x; \theta)$ with p parameters. Define the misfit

$$\chi^2(\theta) = \sum_{i=1}^N \left(\frac{y_i - f(x_i; \theta)}{\sigma_i} \right)^2$$

and choose $\hat{\theta}$ that minimises it.

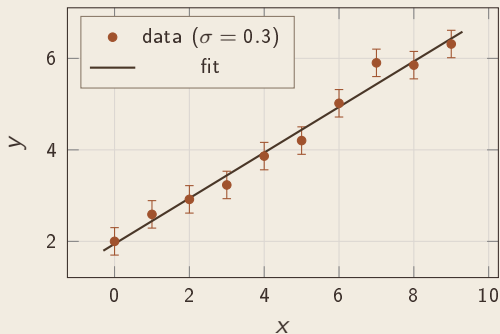
Why squares?

If the errors are independent and Gaussian, minimising χ^2 gives the **maximum-likelihood** parameters.

Why divide by σ_i ?

Each residual is measured in units of its own uncertainty: precise points pull harder, poor points less.

The simplest fit: a straight line



Model $y = a + bx$. Setting $\partial\chi^2/\partial a = \partial\chi^2/\partial b = 0$ gives two **linear** equations: an exact solution, no iteration.

	true	fitted
a	2.0	1.95 ± 0.18
b	0.5	0.499 ± 0.033

$$\chi^2 = 4.2, \nu = N - p = 8$$

Linear least squares in general

“Linear” means linear in the **parameters**, not in x :

$$f(x; \boldsymbol{\theta}) = \sum_{j=1}^p \theta_j \phi_j(x) \quad \text{e.g. } \theta_1 + \theta_2 x + \theta_3 x^2, \quad \theta_1 \sin \omega t + \theta_2 \cos \omega t$$

Design matrix $A_{ij} = \phi_j(x_i)/\sigma_i$, data $b_i = y_i/\sigma_i$:

$$\chi^2 = \|\mathbf{A}\boldsymbol{\theta} - \mathbf{b}\|^2 \quad \Rightarrow \quad \boxed{(\mathbf{A}^T \mathbf{A}) \hat{\boldsymbol{\theta}} = \mathbf{A}^T \mathbf{b}}$$

Normal equations

A $p \times p$ linear system: one solve, exact minimum. Also gives the covariance $\mathbf{C} = (\mathbf{A}^T \mathbf{A})^{-1}$. But solve it with QR/SVD (`np.linalg.lstsq`): forming $\mathbf{A}^T \mathbf{A}$ **squares the condition number** (Lecture 1).

Two meanings of “linear”

	Linear dynamics (Lecture 2)	Linear in parameters (today)
Property of	the system	the fitting function
Meaning	superposition; frequency independent of amplitude	$f = \sum_j \theta_j \phi_j(x)$
How to tell	test it: superposition, amplitude dependence, harmonics, surrogate data	read it off the formula
Consequence	what behaviour to expect	exact solve, or iterate from starting values

They are independent

Harmonic oscillator, $x = A \sin(\omega t + \varphi)$: **linear dynamics**, but fitting an unknown ω is a **nonlinear fit**.
Van der Pol limit cycle: **nonlinear dynamics**, but its Fourier series with known period is a **linear fit**.

Nonlinear models

Parameter inside a function, e.g. $C(t) = C_{ss} + (C_0 - C_{ss}) e^{-t/\tau}$. Now $\chi^2(\boldsymbol{\theta})$ is not quadratic: **iterate**.

Levenberg–Marquardt

Linearise the model around the current $\boldsymbol{\theta}$, solve the linear least-squares problem, repeat. LM adds damping: gradient descent far away, Gauss–Newton near the minimum.

In Python

```
p, C = curve_fit(model, t, y,
                  p0=(0.8, 0.1, 3),
                  sigma=s,
                  absolute_sigma=True)
err = np.sqrt(np.diag(C))
```

Needs a **starting guess** p0: local minima again (Part A).

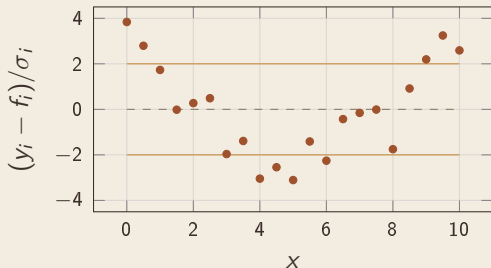
PART C

Is the fit any good?

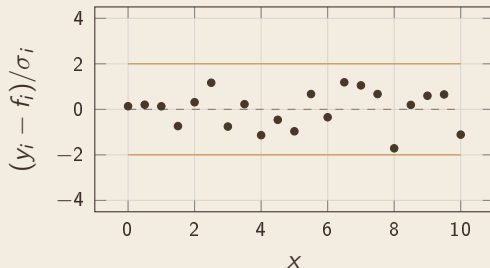
A fitting routine always returns numbers.
Whether they mean anything is your job.

Look at the residuals first

Straight line: $\chi^2/\nu = 4.75$



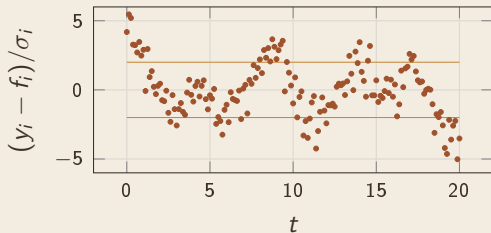
Quadratic: $\chi^2/\nu = 0.76$



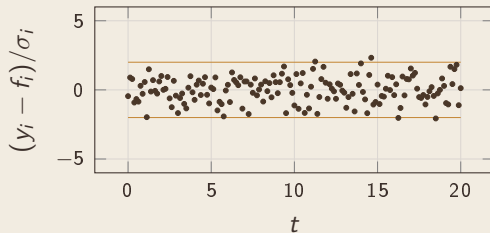
Same data, slightly curved. The line leaves a **systematic arch**: the model is missing dynamics (Lecture 2). The quadratic leaves structureless scatter, mostly within ± 2 .

Residuals: time variability before noise

Constant ω : $\chi^2/\nu = 3.97$



Drifting $\omega(t) = \omega_0 + \beta t$: $\chi^2/\nu = 0.91$



An oscillation whose frequency drifts from 1.0 to 1.3 rad/s, noise $\sigma = 0.1$. The fit recovers $\beta = 0.0153 \pm 0.0007$ (true 0.015).

The tempting fix is wrong

Inflating σ to 0.20 makes the constant- ω fit “acceptable” — and turns deterministic **time variability** into “random error”, doubling every error bar.

The reduced chi-square χ^2/ν

If the model is right and the σ_i are right, each term of χ^2 is about 1, so

$$\boxed{\frac{\chi^2}{\nu} \approx 1 \pm \sqrt{2/\nu}}, \quad \nu = N - p.$$

$\chi^2/\nu \gg 1$ Missing dynamics — often **time variability**. Look at the residuals; do **not** simply inflate σ_i .

$\chi^2/\nu \approx 1$ Consistent: model and uncertainties agree with the data.

$\chi^2/\nu \ll 1$ σ_i overestimated, or **overfitting**. Suspicious, not “excellent”.

Examples so far: drifting ω fitted as constant 3.97; line 4.75 (wrong model); quadratic 0.76; straight-line fit 0.52 with $\nu = 8$ (range 1 ± 0.5).

Parameter uncertainties and correlations

The covariance matrix $C = (A^T A)^{-1}$ (or from `curve_fit`) gives

$$\sigma_{\theta_j} = \sqrt{C_{jj}}, \quad \rho_{jk} = \frac{C_{jk}}{\sqrt{C_{jj} C_{kk}}}.$$

Straight-line example

$a = 1.95 \pm 0.18$, $b = 0.499 \pm 0.033$,
correlation $\rho_{ab} = -0.84$.

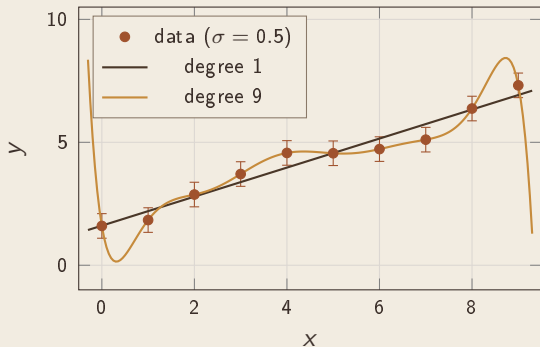
A steeper line with a lower intercept fits almost as well: the errors are **not independent**.

Consequences

Quoting $a \pm \sigma_a$ and $b \pm \sigma_b$ alone overstates what you know about combinations. Centring x (using $x - \bar{x}$) removes the correlation.

$\Delta\chi^2 = 1$ around the minimum defines the 1σ range of one parameter.

Overfitting



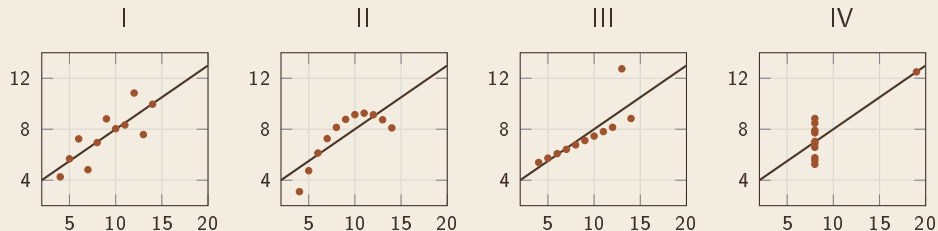
10 points from a straight line plus noise.

degree	ν	χ^2/ν
1	8	0.67
3	6	0.55
6	3	0.06
9	0	— ($\chi^2 = 0$)

Prediction at $x = 10$

Line: 7.5. Degree 9: -72.

Anscombe's quartet: numbers are not enough



All four: mean $\bar{x} = 9$, $\bar{y} = 7.50$, fitted line $y = 3.00 + 0.500x$, $R^2 = 0.67$. Only one is a sensible straight-line fit.

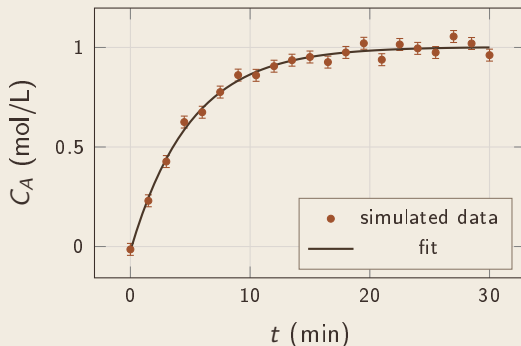
Plot the data and the residuals before you trust a single summary number.

PART D

Generated data and measured data

First test the fitting procedure where you know the answer.
Then use it where you do not.

Generated data: can we recover what we put in?



CSTR relaxation from Lecture 2, plus noise $\sigma = 0.03$, $N = 21$.

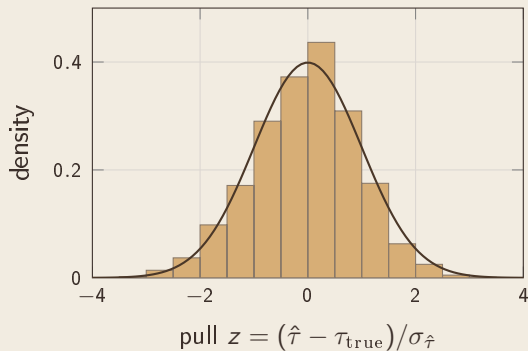
	true	fitted
C_{ss}	1.0	1.003 ± 0.011
C_0	0.0	-0.021 ± 0.026
τ	5.0	5.00 ± 0.27

$$\chi^2/\nu = 0.93$$

Verification of the fit

All three true values lie within 1σ . The procedure works — for *this* realisation.

Are the error bars right?



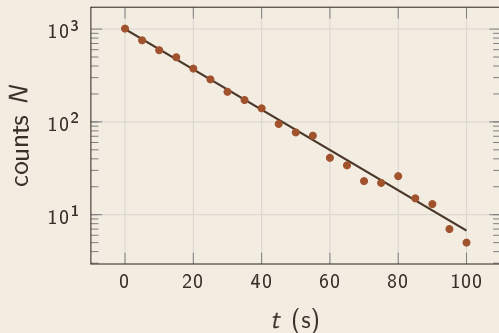
Repeat the experiment 2000 times (Monte Carlo, Lecture 2). For each fit compute the **pull**.

	expected	found
mean	0	-0.04
std	1	0.97
$ z < 1$	68%	70%

Conclusion

No bias, and the quoted $\sigma_{\hat{\tau}}$ are honest.

Measured data: the noise model matters



Radioactive decay, $N = N_0 e^{-t/\tau}$. Counts are **Poisson**: $\sigma_i = \sqrt{N_i}$, not constant. Late points: 15, 13, 7, 5 counts.

Method (2000 data sets, $\tau = 20$ s)	spread of $\hat{\tau}$
Nonlinear fit, weights $1/\sqrt{N_i}$	± 0.34 s
Straight line through $\ln N$, unweighted	± 0.68 s

Lesson

“Linearising” with logs and ignoring weights lets the noisiest points count as much as the best ones: same data, **twice** the uncertainty.

What statistics cannot see

Statistical uncertainty

Random scatter. Shrinks as $1/\sqrt{N}$. Detected by residuals, χ^2 , pulls.

Systematic error

A wrong calibration, model or assumption. Does **not** shrink with more data. Invisible to χ^2 .

Hubble, 1929

A clean linear fit gave $H_0 \approx 500 \text{ km s}^{-1} \text{ Mpc}^{-1}$; today about 70. The distances were systematically wrong — more data would only have made the wrong answer more precise.

A fitting workflow

	Step	Tool / check
1	Plot the data; classify the system (Lecture 2)	Expected shape, noise type
2	Choose the simplest physical model	Linear in θ ? (not the same as linear dynamics)
3	Test on generated data with known θ	Recover θ ; pulls $\sim \mathcal{N}(0, 1)$
4	Fit the measured data	lstsq / curve_fit, honest σ_i , good p0
5	Judge the fit	Residuals: time variability before noise; χ^2/ν
6	Report	$\hat{\theta} \pm \sigma$, correlations, χ^2/ν , systematics

Before you fit: five questions

1. What kind of system produced the data? (*linear or nonlinear dynamics; time-varying?*)
2. Is my fitting function linear in its parameters? (*exact solve, or iterate from starting values*)
3. What is the noise, and do I know σ_i ? (*Gaussian? Poisson? constant?*)
4. Can I recover known parameters from simulated data? (*verification*)
5. Is there time variability left in the residuals, before I call it noise? (*goodness of fit*)

Then report the parameters, their uncertainties and their correlations.

TO TAKE AWAY

A fit always returns numbers.

Know the system, choose the model, test on known data, look for time variability in the residuals — then believe the numbers.